



Problem Statement

Design and partially implement an **end-to-end “Chat with SQL” system** that allows users to ask **natural-language questions** about a **large relational database (100+ tables)** and receive **natural-language answers**.

The system must:

1. Convert a user’s natural-language question into a **SQL query**
2. Execute the SQL query on the database
3. Convert the **query results into a human-readable answer**
4. Use **Retrieval-Augmented Generation (RAG)** to handle large database schemas efficiently

Because the schema is large, the full schema **cannot** be sent to the LLM on every request.

User Question (NL)



Relevant Schema Retrieval (RAG)



SQL Generation (LLM)



SQL Validation & Guardrails



SQL Execution on Database



Query Result Rows



Natural-Language Answer (LLM)

Functional Requirements

1. Natural Language → SQL

Given a user question like:

“Which 5 customers spent the most money last year?”

The system should:

- Identify relevant tables and columns
- Generate a valid, read-only SQL query
- Ensure correctness of joins, filters, and aggregations

2. SQL Execution

- The generated SQL must be **executed against the database**
- You may assume:
 - Read-only access
 - Structured query results (rows + columns)
- Error handling should be considered (invalid SQL, empty results)

3. SQL Result → Natural Language



The system must:

- Take the raw SQL output (rows)
- Convert it into a **clear natural-language answer**

Example:

"The top 5 customers by total spending in 2023 were Alice Corp (\$1.2M), Beta Ltd (\$980k), ..."

Retrieval-Augmented Generation (RAG) Requirements

Because the database has **100+ tables**, the system must retrieve only **relevant schema metadata**, such as:

- Table names
- Table descriptions
- Column names and types
- Foreign-key relationships

You should explain:

- How schema metadata is indexed
- How relevant schema is retrieved per question
- Why this retrieval strategy scales

Database Assumptions

- Relational database (PostgreSQL / MySQL-like)
- Schema metadata is available (e.g., catalog table or JSON)
- Example tables:
 - customers
 - orders
 - products
 - order_items
 - payments

Expected Deliverables

1. High-Level System Design (Required)

Provide a **written architecture description** (diagram optional) explaining:

- Schema indexing & retrieval layer
- SQL generation step
- SQL execution step
- Result-to-text generation step
- Guardrails and validation

2. Pseudocode / Lightweight Implementation (Required)

Provide pseudocode or minimal code for:

a) Schema Retrieval (RAG)

- How schema metadata is stored



- How it is embedded or searched
- How top-K relevant tables/columns are selected

b) SQL Generation Prompt

- How retrieved schema is injected into the prompt
- How the LLM is instructed to:
 - Generate **only SELECT queries**
 - Use only retrieved tables
 - Avoid hallucinated columns

c) SQL Execution Layer

- How generated SQL is executed
- How errors are handled
- How results are structured for downstream use

d) Result → Natural Language Answer

- How SQL results are summarized
- How the LLM converts rows into a human-friendly explanation
- How edge cases are handled (empty results, large result sets)

3. End-to-End Example (Required)

Provide **one complete example** showing all stages.

Example

User Question

"Which 3 products generated the highest revenue in 2024?"

Retrieved Schema

Table: products

- product_id (PK)
- product_name
- price

Table: order_items

- order_id
- product_id (FK → products.product_id)
- quantity
- unit_price

Table: orders

- order_id (PK)
- order_date

Generated SQL

```
SELECT p.product_name,  
       SUM(oi.quantity * oi.unit_price) AS revenue
```



```
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_date BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY p.product_name
ORDER BY revenue DESC
LIMIT 3;
```

SQL Result

product_name	revenue
Product A	2,500,000
Product B	1,900,000
Product C	1,300,000

Final Natural-Language Answer

In 2024, the top three products by revenue were Product A (\$2.5M), followed by Product B (\$1.9M) and Product C (\$1.3M).

Guardrails & Safety (Must Address)

Your design must consider:

- Blocking INSERT, UPDATE, DELETE, DROP
- Ensuring SQL uses **only retrieved schema**
- Handling ambiguous questions (e.g., "top customers" by what metric?)
- Preventing hallucinated columns or joins

Discussion & Reasoning (Required)

Briefly explain:

- Assumptions about the database
- Trade-offs in schema retrieval granularity
- Limitations of LLM-based SQL generation
- Failure cases and mitigation strategies

Constraints

- Any language or LLM framework is allowed
- Emphasis is on **reasoning and design**
- Code does not need to be runnable
- Should comfortably fit in **60–90 minutes**

Evaluation Criteria



Candidates will be evaluated on:

1. **Correct end-to-end reasoning**
 - NL → SQL → DB → NL
2. **RAG correctness**
 - What schema is retrieved and why
3. **SQL quality**
 - Joins, filters, aggregations, correctness
4. **Result interpretation**
 - Turning rows into clear explanations
5. **Risk awareness**
 - Hallucinations, safety, ambiguity, scalability